

Zadanie 1 Najpierw spróbujemy rozwiązać inne zadanie - dla danych $T[], n, k$ odpowiedzieć, czy istnieje takie rozłożenie punktów dostępu, że każda klasa jest odległa od swojego najbliższego punktu dostępu o co najwyżej d , gdzie d jest jakąś liczbą całkowitą z przedziału $[1, T[n] - T[0]]$.

Rozwiązania tego zadania mają własność monotoniczności, to znaczy jeśli sprawdzimy, że dla jakiegoś dystansu d nie istnieje ustawienie punktów spełniające powyższe założenia, to od razu wiemy, że nie istnieje też żadne ustawienie dla wszystkich $d' < d$, bo każde takie ułożenie, gdyby istniało, byłoby też dobre dla d . Jeśli dla jakiegoś dystansu d dobre ułożenie punktów istnieje, to aby sprawdzić czy jest ono najbardziej optymalnym, to sprawdzamy, czy dla jakiegokolwiek $d' < d$ dobre ustawienie istnieje.

Zatem możemy przeszukiwać binarnie przedział $[1, T[n] - T[0]]$, co będzie się wykonywało (pesymistycznie) $\Theta(\log c)$ razy, gdzie $c = T[n] - T[0]$, czyli długość przeszukiwanego przedziału. Po wyszukiwaniu dostaniemy odpowiedź na nasze oryginalnie postawione pytanie, bo dostaniemy najbardziej optymalne ustawienie punktów dostępu i największy (*osiągany!* bo gdyby nie był, to rozłożenie punktów działałoby również dla dystansu $d - 1$) dystans punktu dostępu do którejś z klas.

Teraz ustalmy $d \in [1, T[n] - T[0]]$ i będziemy dla niego sprawdzać, czy istnieje dobre rozstawienie punktów dostępu. Potrzebujemy jeszcze zmiennej pomocniczej **pom**, która będzie przechowywała koordynat ostatnio postawionego punktu dostępu. Inicjalizujemy **pom** = $T[0] + d$, bo Pierwszy punkt dostępu położymy właśnie tam. Rozpoczynamy pętlę, która ma wykonywać się $k-1$ razy (lub mniej). i -ta iteracja pętli przechodzi w następujący sposób. Przeszukujemy binarnie tablicę $T[n]$ szukając pierwszej wartości większej od **pom** + d , czyli pierwszej klasy, której dotychczasowe ułożenie punktów dostępu nie obejmuje w odległości d - złożoność (pesymistyczna) $\Theta(\log n)$.

Jeśli takiej wartości nie ma, to otrzymaliśmy odpowiedź, że dla odległości d istnieje dobre ułożenie i kończymy pętlę oraz zwracamy odpowiedź pozytywną dla d . Jeśli taka wartość istnieje, niech to będzie v (tak jak w pesymistycznym przypadku - bo wydłuża działanie programu), to ustawiamy punkt dostępu na koordynat $v + d$ obejmując element v i ewentualnie jakieś inne po nim. Aktualizujemy wartość **pom** = $v + d$, i przechodzimy do następnej iteracji.

Jeśli po wykonaniu całej pętli nie zwróciliśmy odpowiedzi dla d , to wykonujemy ostatnie wyszukiwanie binarne na $T[n]$, które sprawdza, czy otrzymane ustawienie jest poprawne. Znowu szukamy pierwszej wartości w $T[n]$ większej niż **pom** + d - złożoność (pesymistyczna) $\Theta(\log n)$. Jeśli taka wartość istnieje, to zwracamy odpowiedź negatywną, że nie istnieje dobre ustawienie punktów dla d , a jeśli taka wartość *nie* istnieje to zwracamy pozytywną odpowiedź.

W pesymistycznym przypadku ta pętla będzie wykonywać się $k - 1$ razy, a wyszukiujemy wtedy binarnie jeszcze jeden raz, zatem złożoność pesymistyczna całej pętli to $\Theta(k \log n)$.

Takie sprawdzenie dla d będziemy wykonywać (pesymistycznie) $\Theta(\log c)$ razy, co sprawia, że złożoność całego algorytmu to $\Theta(\log c \cdot k \log n)$.